The Journal of Open Source Software

ollamar: An R package for running large language models

Hause Lin [1] **and Tawab Safi** [1]

1 Massachusetts Institute of Technology, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Chris Vernon

Reviewers:

- @KennethEnevoldsen
- @elenlefoll

Submitted: 24 August 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Large language models (LLMs) have transformed natural language processing and AI. Many tools like Ollama (https://ollama.com/) have been developed to allow users to easily deploy and interact with LLMs hosted on users' own machines. `ollamar` is an R library that interfaces with Ollama, allowing R users to easily run and interact with LLMs. This library is valuable for researchers and data scientists integrating LLMs into R workflows. `ollamar` is actively developed on GitHub (https://github.com/hauselin/ollamar) and available on the Comprehensive R Archive Network (https://cran.r-project.org/web/packages/ollamar/index.html).

Statement of Need

The increasing importance of LLMs in various fields has created a demand for accessible tools that allow researchers and practitioners to leverage LLMs within their preferred programming environments. Locally deployed LLMs offer advantages in terms of data privacy, security, and customization, making them an attractive option for many users (Chan et al., 2024; Liu et al., 2024; Lytvyn, 2024; Shostack, 2024). However, the lack of native R libraries for interfacing with locally deployed LLMs has limited the accessibility of these models to R users, even though R is a popular and crucial tool in statistics, data science, and various research domains (Hill et al., 2024). `ollamar` fills a critical gap in the R ecosystem by providing a native interface to run locally deployed LLMs, and is already being used by researchers and practitioners (Turner, 2024).

The `ollamar` R library is a package that integrates R with Ollama, allowing users to run large language models locally on their machines. Although alternative R libraries exist (Gruber & Weber, 2024), `ollamar` distinguishes itself through the features described below.

User-friendly design: It provides an interface to the Ollama server and all API endpoints, closely following the official API design. This design makes it easy for R users to understand how similar libraries (such as in Python and JavaScript) work while allowing users familiar with other programming languages to adapt to and use this library quickly. The consistent API structure across languages facilitates seamless transitions and knowledge transfer for developers working in multi-language environments.

Consistent and flexible output formats: All functions that call API endpoints return `httr2::httr2_response` objects by default, but users can specify different output formats, such as dataframes (`"df"`), lists (of JSON objects) (`"jsonlist"`), raw strings (`"raw"`), text vectors (`"text"`), or request objects (`"req"`). This flexibility greatly enhances the usability and versatility of the library. Users can choose the format that best suits their needs, such as when working with different data structures, integrating the output with other R packages, or allowing parallelization via the `httr2` library.

Easy management of LLM conversation history: LLM APIs often expect conversation/chat

history data as input, often nested lists or JSON objects. Note that this data format is standard for chat-based applications and APIs (not limited to Ollama), such as those provided by OpenAI and Anthropic. `ollamar` simplifies preparing and processing conversational data for input to different LLMs, streamlining the workflow for the most popular chat-based applications.

```
# nested list of chat history with multiple messages
list(
    list(role = "system", content = "Be kind."),
    list(role = "user", content = "Hi! How are you?")
)
```

Usage and examples

This section highlights the key features of `ollamar`. For documentation and detailed examples, see https://hauselin.github.io/ollama-r/.

Install and use Ollama

1. Download and install Ollama from https://ollama.com
2. Open/launch the Ollama app to start the local server
3. Install `ollamar` in R by running `install.packages("ollamar")`

```
install.packages("ollamar")
library(ollamar)  # load ollamar

test_connection()  # test connection to Ollama server
# <httr2_response>
# GET http://localhost:11434/
# Status: 200 OK  # indicates connected to server
```

Manage LLMs

To use Ollama, you must first download the model you want to use from https://ollama.com/library. All examples below use the Google's Gemma 2 LLM (specifically, the 2-billion parameter model, which is about 1.6GB, as of August 2024).

```
# download model, https://ollama.com/library/gemma2:2b
pull("gemma2:2b")

# two ways to verify it's downloaded
list_models()
model_avail("gemma2:2b")
```

Call API endpoints

`ollamar` has distinct functions for each official Ollama API endpoint (see https://hauselin.github.io/ollama-r/reference/index.html). By default, all functions calling API endpoints will return an `httr2::httr2_response` object (see https://httr2.r-lib.org/index.html). You can then parse/process the response object using the `resp_process()` function.

```
# generate text based on a single prompt
resp <- generate("gemma2:2b", "tell me a 5-word story")
resp_process(resp, "text")
resp_process(resp, "df")
resp_process(resp, "jsonlist")
resp_process(resp, "raw")
```

DRAFT

```r
# generate text based on chat or conversation history
# create messages in a chat history
messages <- create_messages(
  create_message("end all your sentences with !!!", role = "system"),
  create_message("Hello")  # default role is user
)
resp <- chat("gemma2:2b", messages)  # make request with chat API endpoint

# get vector embedding for prompts
embed("gemma2:2b", "Hello, how are you?")
embed("gemma2:2b", c("Hello, how are you?", "Good bye"))
```

Manage chat history

When chatting with a model, Ollama and other LLM providers like OpenAI and Anthropic require chat/conversation histories to be formatted in a particular way. ‘ollamar makes it easy to manipulate the messages in the chat history.

```r
# initialize or create messages for a chat history
messages <- create_messages(
  create_message("end all your sentences with !!!", role = "system"),
  create_message("Hello")  # default role is user
)

# add message to the end of chat history
messages <- append_message("Hi, how are you?", "assistant", messages)
# delete message at index/position 1
messages <- delete_message(messages, 1)
# prepend message to the beginning of chat history
messages <- prepend_message("Start all sentences with Yo!", "user", messages)
# insert message at position 2
messages <- insert_message("Yo!", "assistant", messages, 2)
```

Make parallel requests

`ollamar` uses the `httr2` library, which provides functions to make parallel requests. Below is a simple example demonstrating how to perform sentiment analysis in parallel. Specifically, we use the `generate()` function with the parameter `output = "req"`, which asks the function to return an `httr2::httr2_request` object instead of making the request.

```r
library(httr2)

texts_to_classify <- c(
    'I love this product',
    'I hate this product',
    'I am neutral about this product',
    'I like this product'
)

# create httr2_request objects for each text with the same system prompt
reqs <- lapply(texts_to_classify, function(text) {
  prompt <- paste0("Is the statement positive, negative, or neutral? ", text)
  generate("gemma2:2b", prompt, output = "req")
})
```

DRAFT

```r
# make parallel requests and get responses
resps <- req_perform_parallel(reqs)

# process each response with resp_process to extract text
sapply(resps, resp_process, "text")
```

Conclusion

`ollamar` bridges a crucial gap in the R ecosystem by providing seamless access to large language models through Ollama. Its user-friendly API, flexible output formats, and conversation management functions enable R users to integrate LLMs into their workflows easily. This library empowers researchers and data scientists across various disciplines to leverage the power of locally deployed LLMs, potentially accelerating research and development in fields relying on R for data analysis and machine learning.

Acknowledgements

This project was partially supported by the Canadian Social Sciences & Humanities Research Council Tri-Agency Funding (funding reference: 192324).

References

Chan, R. S.-Y., Nanni, F., Brown, E., Chapman, E., Williams, A. R., Bright, J., & Gabasova, E. (2024). Prompto: An open source library for asynchronous querying of LLM endpoints. *arXiv*. https://doi.org/10.48550/arXiv.2408.11847

Gruber, J. B., & Weber, M. (2024). rollama: An R package for using generative large language models through Ollama. *arXiv*. https://doi.org/10.48550/arXiv.2404.07654

Hill, C., Du, L., Johnson, M., & McCullough, B. D. (2024). Comparing programming languages for data analytics: Accuracy of estimation in Python and R. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, *14*(3), e1531. https://doi.org/10.1002/widm.1531

Liu, F., Kang, Z., & Han, X. (2024). Optimizing RAG techniques for automotive industry PDF chatbots: A case study with locally deployed Ollama models. *arXiv*. https://doi.org/10.48550/arXiv.2408.05933

Lytvyn, O. (2024). Enhancing propaganda detection with open source language models: A comparative study. *Proceedings of the MEi:CogSci Conference*, *18*(1). https://journals.phl.univie.ac.at/meicogsci/article/view/822

Shostack, A. (2024). The boy who survived: Removing harry potter from an llm is harder than reported. *arXiv*. https://doi.org/10.48550/arXiv.2403.12082

Turner, S. D. (2024). biorecap: an R package for summarizing bioRxiv preprints with a local LLM. *arXiv*. https://doi.org/10.48550/arXiv.2408.11707

DRAFT